K.R. MANGALAM UNIVERSITY

SMART CITY TRAFFIC MANAGEMENT SYSTEM

DAA Capstone Project — Final Report

Student Name	Harshit Kumar
Roll Number	2401201057
Programme	BCA (AI & Data Science)
Semester	V
Course Code	ENCA351
Course Name	Design and Analysis of Algorithms Lab
Faculty	Dr. Aarti
Academic Session	2026–27

Route Planning • Traffic Analysis • Connectivity • Infrastructure Optimization

1. Abstract

The Smart City Traffic Management System is a Python-based educational prototype demonstrating how graph algorithms support road-network planning. Intersections are vertices and roads are weighted edges. The Streamlit application provides road-network management, shortest-route planning, simulated traffic changes, connectivity analysis, minimum spanning tree planning and algorithm benchmarking.

The screenshots in this report show the current application state, including 7 intersections and 13 roads on the dashboard. The benchmarking module reports execution time and peak memory for selected graph algorithms.

2. Introduction and Problem Statement

Urban road networks contain intersections connected by roads with different travel costs. Congestion and road closures can change which route is most efficient. Graph theory models intersections as vertices, roads as edges and road weights as travel or infrastructure costs.

Problem statement: Develop an interactive prototype to model a city road network and demonstrate shortest-route planning, simulated congestion analysis, network connectivity, minimum spanning tree planning and algorithm-performance comparison.

3. Objectives

• Represent intersections and roads using a weighted graph.

• Calculate a minimum-cost route between a selected source and destination.

• Simulate congestion and road disruption by changing road costs or availability.

• Use BFS and DFS to inspect traversal order and reachability.

• Calculate an MST using Prim’s and Kruskal’s algorithms.

• Compare execution time and peak memory across graph algorithms.

• Present results through an interactive Streamlit interface.

4. Technologies Used

Technology	Purpose
Python	Core programming language and algorithm integration.
Streamlit	Interactive browser interface, navigation, forms and results.
NetworkX	Graph representation and operations.
Pandas	Benchmark tables and CSV data handling.
Matplotlib	Graph and performance visualizations.
Jupyter Notebook	Algorithm demonstrations and analysis.
Git / GitHub	Version control and project hosting.

5. System Design and Modules

The application is organized into separate files so the user interface and algorithm logic remain manageable. The Streamlit app provides navigation and input controls, while helper modules perform graph operations.

File	Responsibility
app.py	Streamlit navigation, input controls, graph rendering and results.
route_planner.py	Shortest-route calculation using Dijkstra's algorithm.
traffic_analysis.py	Route and cost comparison after simulated traffic changes.
connectivity_module.py	Connectivity checks, BFS/DFS traversal and connected components.
mst_planner.py	Minimum Spanning Tree using Prim or Kruskal.
benchmark.py	Algorithm execution-time and memory benchmarking.
project_notebook.ipynb	Algorithm demonstrations and experimental analysis.
requirements.txt	Python package dependencies.
README.md	Project overview and setup instructions.

6. Algorithms Implemented

Algorithm	Purpose	Typical complexity
Dijkstra	Minimum-cost route with non-negative weights.	O((V + E) log V)
Bellman–Ford	Shortest paths; supports negative weights if no negative cycle exists.	O(VE)
BFS	Breadth-first traversal and reachability.	O(V + E)
DFS	Depth-first traversal and connectivity exploration.	O(V + E)
Prim	Minimum Spanning Tree using low-cost connecting edges.	O(E log V)
Kruskal	MST by considering edges in ascending weight order.	O(E log E)

V = number of vertices; E = number of edges. Actual performance depends on implementation and graph structure.

7. Application Workflow

• Load the default city graph or add intersections and roads.

• Choose a module from the application navigation.

• Select the required source, destination, road or algorithm.

• Execute the operation and inspect the displayed result.

• Review graph visualizations and export benchmark results when required.

8. Application Screenshots and Outputs

The following figures are captured from the running Streamlit application and demonstrate its main modules.

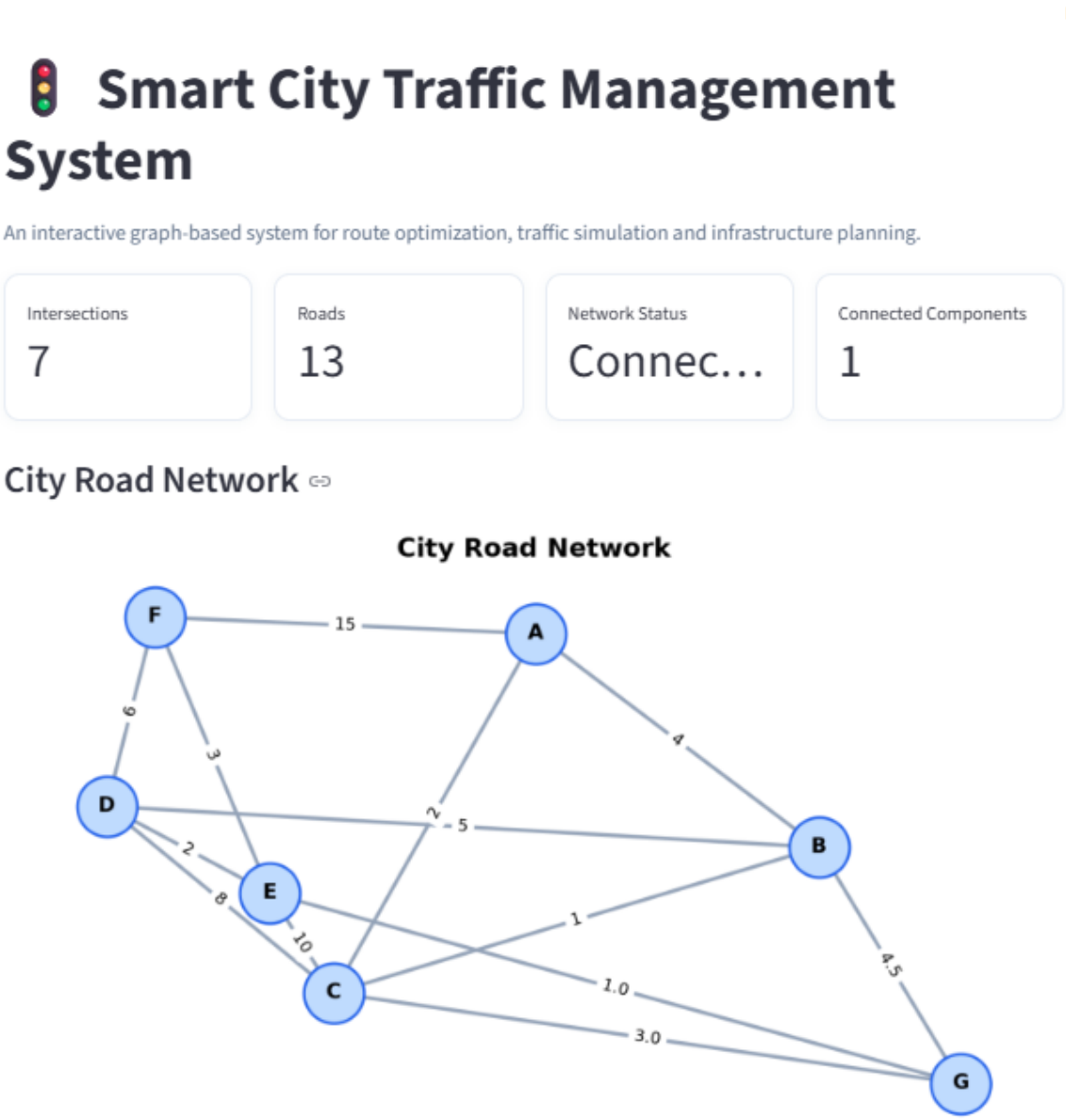


Figure 1. Dashboard: 7 intersections, 13 roads and one connected component.

Deploy

Road Network Management

Add intersections and roads to customize the city graph.

Intersection name

Example: G

+ Add Intersection

Connect two intersections

Starting intersection

A

Ending intersection

B

Road cost / distance

1.00

+ Add / Update Road

Current Road Network

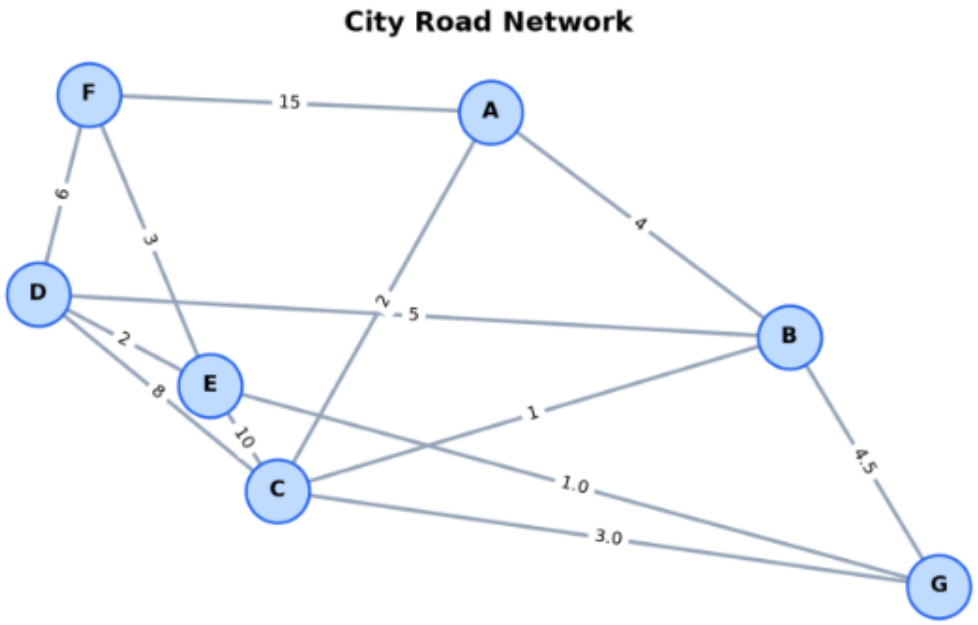


Figure 2. Road Network Management: add intersections and connect roads.

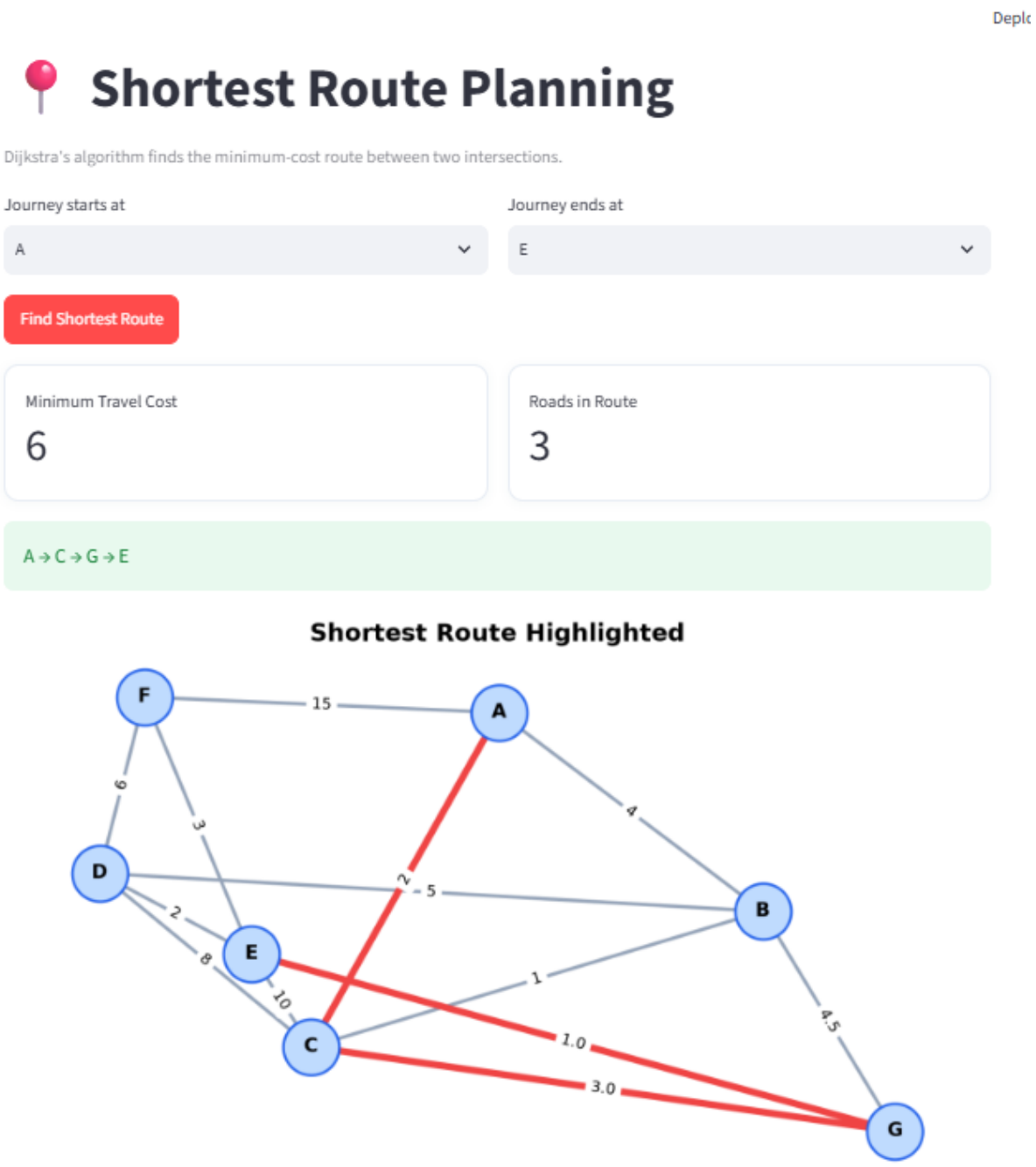


Figure 3. Dijkstra route example: A → C → G → E; minimum travel cost 6.

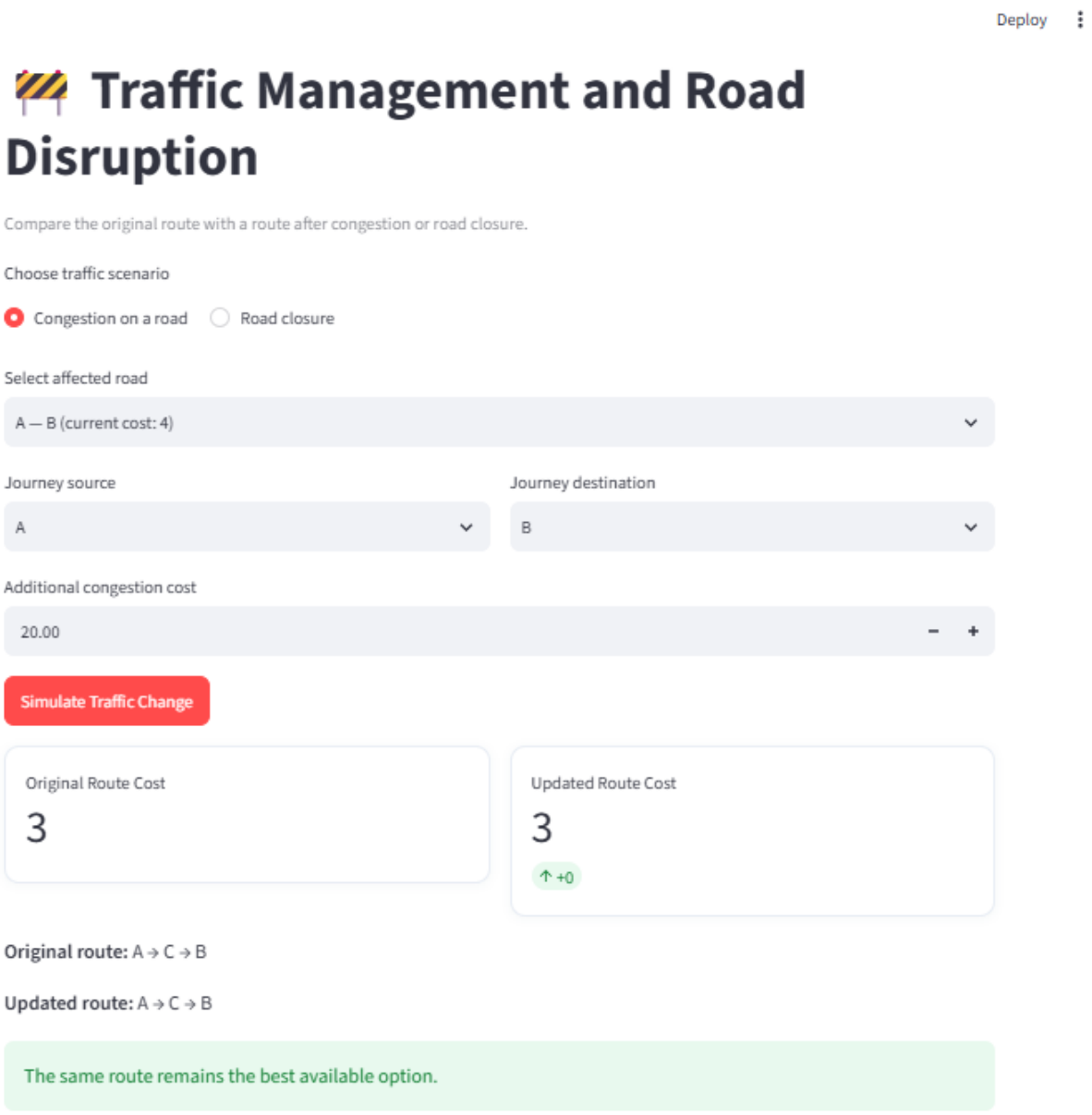


Figure 4. Traffic congestion simulation with original and updated route costs.

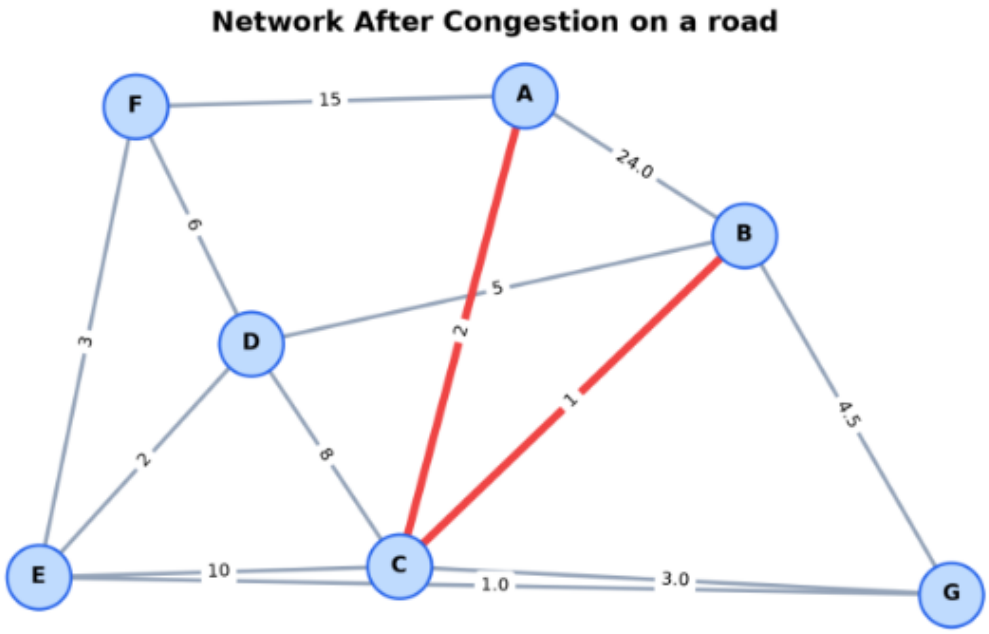


Figure 5. Road-network visualization after a traffic-cost change.

Deploy

🔗 Network Connectivity

Explore graph traversal and connectivity using BFS and DFS.

Network Connected	Connected Components	Total Intersections
Yes	1	7

Traversal starting intersection

A

Run BFS | Run DFS

BFS Traversal

A → B → C → F → D → G → E

Intersections reached: 7

Connected Components

Component 1: A, B, C, D, E, F, G

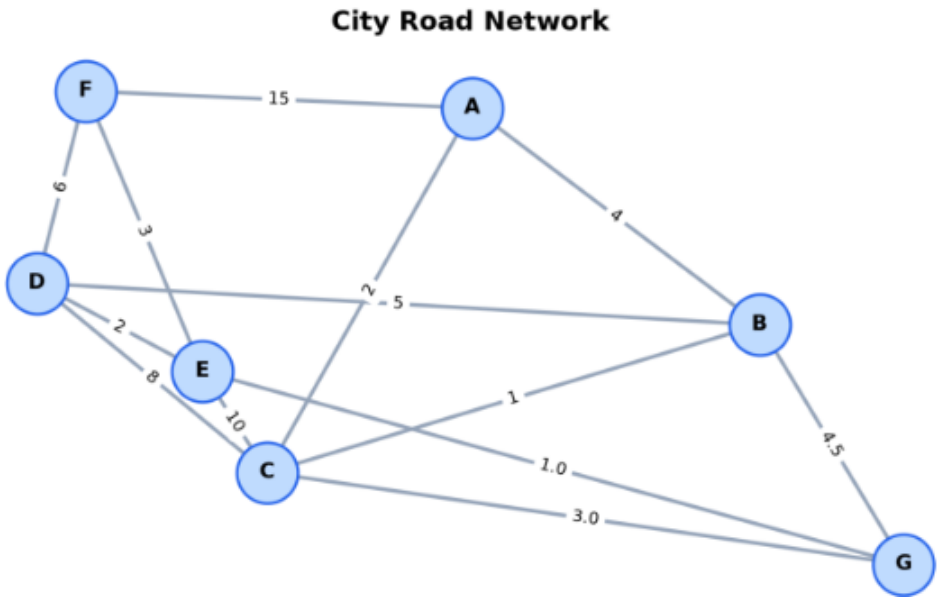


Figure 6. BFS traversal and connected-components output.

De

Network Connectivity

Explore graph traversal and connectivity using BFS and DFS.

Network Connected	Connected Components	Total Intersections
Yes	1	7

Traversal starting intersection

F

Run BFS | Run DFS

DFS Traversal

F → D → B → A → C → E → G

Intersections reached: 7

Connected Components

Component 1: A, B, C, D, E, F, G

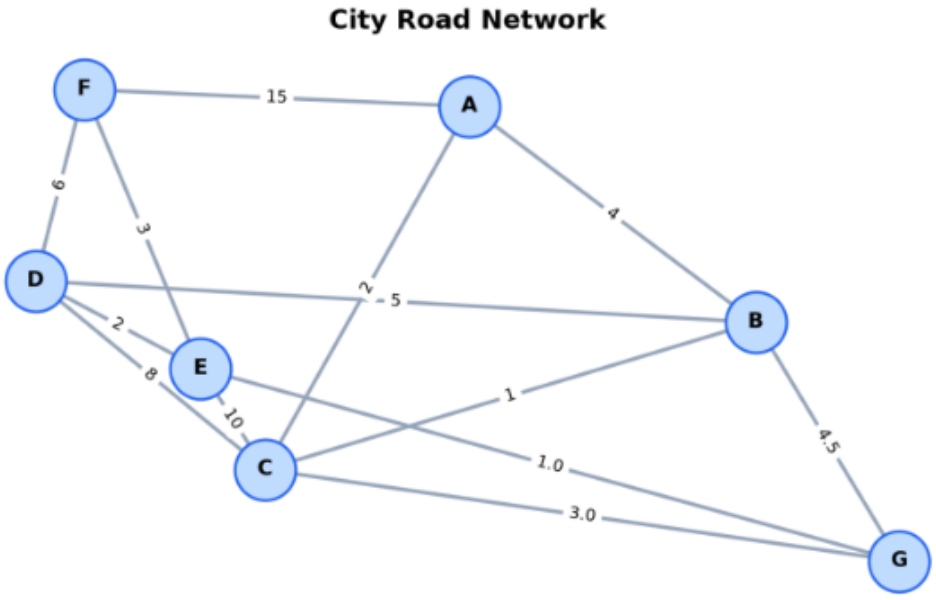


Figure 7. DFS traversal from the selected starting intersection.

Deploy

Infrastructure Planning

Find a Minimum Spanning Tree using Prim's or Kruskal's algorithm.

Select algorithm

Prim

Total Infrastructure Cost

12

Selected Roads

Road	Cost
A — C	2
B — C	1
C — G	3
D — E	2
E — G	1
E — F	3

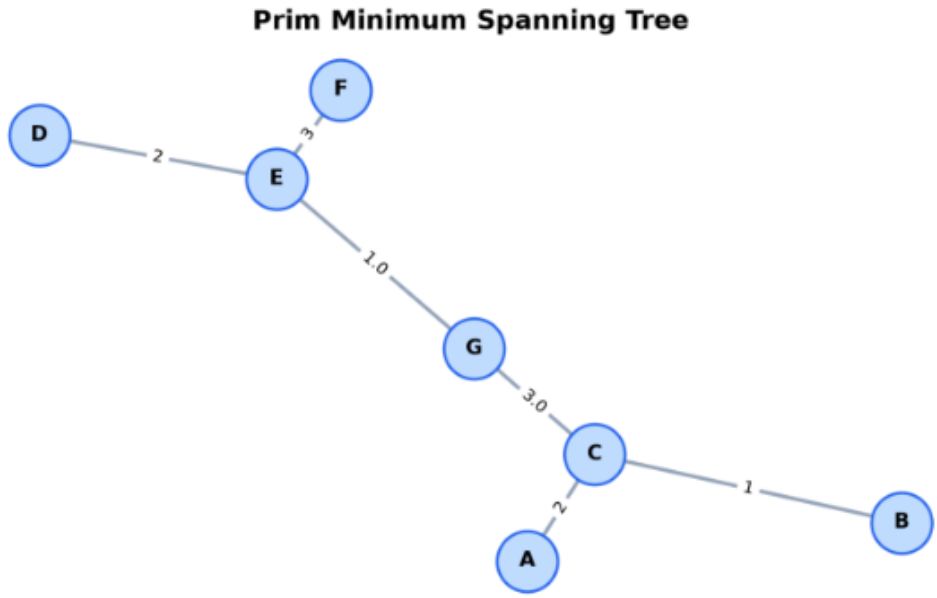


Figure 8. Prim's Minimum Spanning Tree; displayed total infrastructure cost 12.

Deploy

Infrastructure Planning

Find a Minimum Spanning Tree using Prim's or Kruskal's algorithm.

Select algorithm

Kruskal

Total Infrastructure Cost

12

Selected Roads

Road	Cost
A — C	2
B — C	1
C — G	3
D — E	2
E — G	1
E — F	3

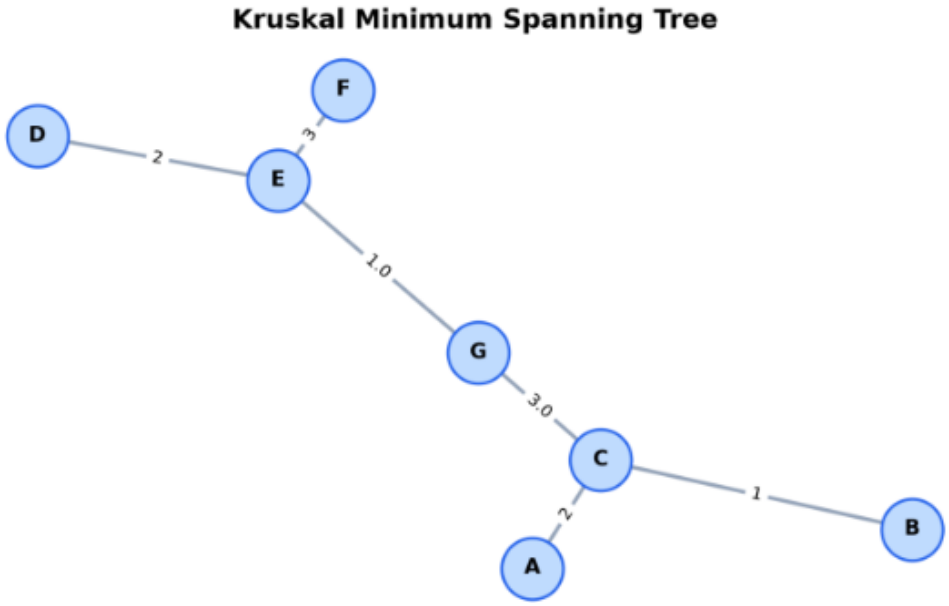


Figure 9. Kruskal's Minimum Spanning Tree; displayed total infrastructure cost 12.

Algorithm Performance Analysis

Compare execution time for BFS, DFS, Dijkstra, Bellman-Ford, Prim and Kruskal.

The benchmark tests networks with different numbers of intersections. Execution time can vary between runs and depends on the computer.

Run Benchmark

Benchmark completed.

Intersections	Roads	Algorithm	Execution Time (ms)	Peak Memory (KB)
10	30	BFS	23.1279	57.95
10	30	DFS	2.9511	48.75
10	30	Dijkstra	3.8636	53.21
10	30	Bellman-Ford	4.9345	51.04
10	30	Prim	0.764	9.76
10	30	Kruskal	2.4104	45.48
25	75	BFS	0.605	20.01
25	75	DFS	0.5551	21.62
25	75	Dijkstra	0.2898	4.49
25	75	Bellman-Ford	0.7572	8.71

Figure 10. Benchmark table with execution-time and peak-memory measurements.

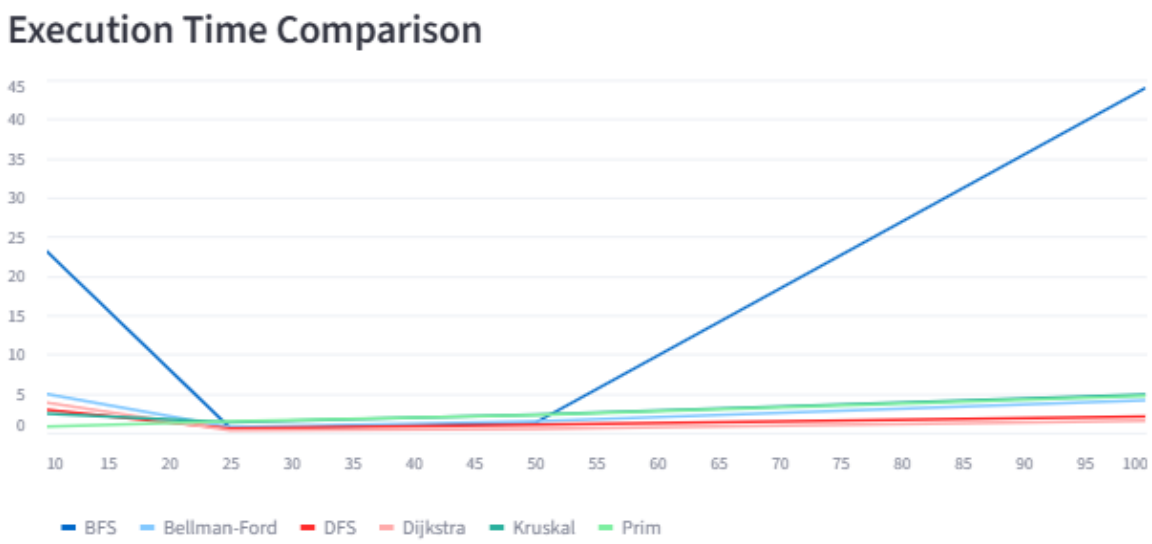


Figure 11. Execution-time comparison chart.

9. Results and Testing

The uploaded application screenshots provide evidence of the following visible results:

Module	Observed result	Demonstration
Dashboard	7 intersections, 13 roads, 1 connected component.	Current network overview.
Shortest route	A → C → G → E; cost 6; 3 roads.	Dijkstra route planning and highlighting.
Traffic management	Original and updated route costs are displayed.	Traffic-cost change comparison.
Connectivity	Connected = Yes; 1 component; 7 intersections.	BFS/DFS and connectivity status.
Prim / Kruskal	Both display total infrastructure cost 12.	MST planning using two algorithms.
Benchmarking	Execution time and peak-memory columns are shown.	Algorithm performance measurement.

10. Performance Analysis

The benchmarking module compares BFS, DFS, Dijkstra, Bellman–Ford, Prim and Kruskal on generated graphs of different sizes. The visible benchmark table includes intersection count, road count, algorithm, execution time in milliseconds and peak memory in kilobytes. A chart visualizes execution-time trends.

Benchmark values can vary between runs because of system load, interpreter overhead and the short duration of operations on small graphs. They should be treated as measurements from the current environment rather than universal rankings. Repeated trials on identical graph instances would provide a more robust comparison.

11. Limitations and Future Enhancements

Current limitations

• The network is illustrative and does not use real geographic coordinates.

• Road weights represent generalized costs rather than calibrated live travel times.

• Traffic changes are simulated manually rather than obtained from a live traffic service.

• Benchmark results depend on the local computer and generated graph instances.

Future enhancements

• Integrate real road-network data and map-based visualization.

• Connect to live traffic services to update road costs dynamically.

• Store network data and benchmark runs in a database.

• Repeat benchmark trials and report average, median and standard deviation.

• Add automated tests for disconnected graphs and invalid inputs.

12. Conclusion

The Smart City Traffic Management System demonstrates how graph algorithms can support route planning, simulated traffic analysis, connectivity inspection and infrastructure planning. The Streamlit interface brings these tasks together in an interactive prototype, while benchmarking provides execution-time and memory comparisons. The submitted screenshots show the application’s principal features and outputs. The project provides a foundation for future work using real road data and dynamic traffic costs.

13. References

• Python Documentation — https://docs.python.org/

• Streamlit Documentation — https://docs.streamlit.io/

• NetworkX Documentation — https://networkx.org/documentation/stable/

• Pandas Documentation — https://pandas.pydata.org/docs/

• Matplotlib Documentation — https://matplotlib.org/stable/

PROJECT GITHUB REPOSITORY

https://github.com/Harshitkumar0018/smart-city-traffic-system-Harshit-Kumar/tree/main